

移动应用开发实验报告

封面信息

项目	内容
实验名称	跨平台综合项目实战
实验编号	d20301035106、d20301009206
学号	2023210621
姓名	史东振
班级	软件工程232
日期	2026年5月15日
组别	2
项目名称	二手物品交易平台 (CG2-SGTPDI)
负责技术栈	HarmonyOS (ArkTS)

实验目的

- 掌握HarmonyOS应用开发能力，熟练使用ArkTS声明式UI和ArkUI组件
- 体验Git团队协作流程，掌握分支管理和代码提交流程
- 学会AI辅助开发 (TRAE)，提高开发效率
- 完成跨平台技术选型分析，理解各平台优劣
- 实现分布式设备协同、AI信用评级、AI议价等特色功能

实验环境

技术栈	开发工具	语言	主要依赖
HarmonyOS	DevEco Studio 5.0	ArkTS	@kit.ArkUI, @kit.NetworkKit, @kit.CoreParkKit
Node.js	VS Code	JavaScript	Express, cors, multer
SQLite	sql.js	SQL	嵌入式关系型数据库

第一部分：项目规划与需求分析

1. 项目选题

- [√] 二手物品交易平台

2. 功能需求

功能模块	具体需求	技术实现
用户登录/注册	用户名密码登录、注册、角色区分（用户/管理员）	Index.ets + ApiService
商品浏览	二手商品列表展示、分类筛选、搜索	ProductList.ets
商品详情	商品图片、价格、描述、卖家信息	ProductDetail.ets
购物车	添加商品、修改数量、删除、结算	Cart.ets
下单交易	选择地址、确认订单、提交订单	CreateOrder.ets
交易管理	我的交易列表、交易详情、状态跟踪	MyTrades.ets + TradeDetail.ets
AI信用评估	基于GLM/DeepSeek/豆包多引擎的信用评分	CreditEvaluation.ets + AIService
AI智能议价	AI分析商品行情、辅助买卖双方议价	ProductDetail.ets + apiBargain
AI智能客服	AI助手实时解答用户问题	AiAssistant.ets
设备协同	分布式设备扫描、交易数据同步	DeviceSynergy.ets + DistributedService
好友管理	添加好友、聊天通讯	FriendList.ets + Chat.ets
商品发布	图片上传、商品信息填写、AI识图	Productization.ets
数据安全	交易数据加密保护、权限管理	DataSecurity.ets
通知推送	系统通知、交易通知、消息提醒	Notification.ets
数据统计	平台交易数据总览（管理员）	DataStatistics.ets
用户管理	用户信息管理（管理员）	UserManagement.ets
管理员交易管理	平台所有交易记录查看和管理	AdminTradeManage.ets
地址管理	收货地址增删改查、默认地址设置	AddressManage.ets
数据库查看	本地数据库数据可视化	DatabaseViewer.ets
个人中心	个人信息管理、头像、昵称修改	Profile.ets

3. 团队分工

本团队共6人 分别负责6个不同技术栈的开发：

成员	技术栈	负责模块
史东振	HarmonyOS (ArkTS)	鸿蒙端全部22个页面+ 5个Service模块开发
庞勇涛	Android (Java)	Android端功能开发
张政	Flutter (Dart)	Flutter端功能开发
张海天	MAUI (C#)	MAUI端功能开发
倪晨鸣	Cordova (HTML/JS)	Cordova端功能开发
方景堂	微信小程序	小程序端功能开发

后端服务 (Express + SQLite) 由团队共同维护。

第二部分：Git团队协作

1. Git仓库构建

```
# 克隆项目仓库
git clone https://gitee.com/chzuczldl/cg2-sgtpdi.git
cd cg2-sgtpdi

# 创建个人开发分支
git checkout -b feat-sdz

# 创建功能分支（分阶段开发）
git checkout -b feature/login
git checkout -b feature/product-list
git checkout -b feature/ai-credit
git checkout -b feature/device-synergy
git checkout -b feature/cart
git checkout -b feature/chat
git checkout -b feature/distributed
git checkout -b feature/security
```

2. .gitignore配置

```
# HarmonyOS项目忽略文件
node_modules/
.idea/
build/
```

```
*.har
*.apk
.DS_Store
hvigor/
.vs/
```

3. 代码提交规范

- 提交信息格式: [功能模块] 具体修改内容
- 定期拉取最新代码: `git pull origin feat-sdz`
- 提交前进行代码审查
- 示例提交记录:
 - [登录] 实现用户登录页面和API对接
 - [AI评估] 集成GLM/DeepSeek/讯飞三模型信用评估
 - [设备协同] 完成设备扫描和同步开关功能
 - [购物车] 实现购物车增删改查和结算

4. 实际提交记录

```
dba517a 添加AI功能、议价系统、管理员交易管理、设备协同优化
f84098f 添加信用评估完整报告查看功能和API Key配置
628e222 修复编译错误: Refresh组件语法和showDialog问题
c7d49c0 添加商品图片功能: 发布商品支持图片URL、商品列表和详情页显示图片
082a901 修复标题被遮挡: 增加背景高度至300, 登录卡片下移至240
```

第三部分: HarmonyOS实现

1. 项目结构

```
entry/src/main/ets/
├─ entryability/
│   └─ EntryAbility.ets          # 应用入口, 管理生命周期
├─ entrybackupability/
│   └─ EntryBackupAbility.ets   # 备份能力
├─ pages/                       # 页面组件 (22个)
└─ └─ Index.ets                 # 登录/注册页面
```

```

|   |— HomePage.ets           # 首页（功能模块入口，区分管理员/用户）
|   |— ProductList.ets        # 商品列表（分类筛选、搜索）
|   |— ProductDetail.ets      # 商品详情（含AI议价）
|   |— Cart.ets               # 购物车
|   |— CreateOrder.ets        # 创建订单
|   |— MyTrades.ets           # 我的交易
|   |— TradeDetail.ets        # 交易详情
|   |— Profile.ets            # 个人中心
|   |— AddressManage.ets      # 地址管理
|   |— Chat.ets               # 聊天
|   |— AiAssistant.ets        # AI助手
|   |— DeviceSynergy.ets      # 设备协同
|   |— CreditEvaluation.ets    # AI信用评估
|   |— Notification.ets       # 通知中心
|   |— FriendList.ets         # 好友列表
|   |— AdminTradeManage.ets    # 管理员交易管理
|   |— DatabaseViewer.ets     # 数据库查看
|   |— DataSecurity.ets       # 数据安全
|   |— DataStatistics.ets     # 数据统计（管理员）
|   |— Productization.ets     # 发布商品
|   |— UserManagement.ets     # 用户管理（管理员）
|   |— common/services/      # 服务模块（5个）
|       |— ApiService.ets     # API请求封装（HTTP通信）
|       |— AIService.ets      # AI服务（GLM/DeepSeek/豆包多引擎）
|       |— DatabaseService.ets # 本地数据库服务
|       |— SimpleDatabaseService.ets # 简化数据库服务
|       |— DistributedService.ets # 分布式服务（设备协同）

```

2. 核心功能实现

2.1 用户登录/注册模块

```

// Index.ets - 登录页面核心
import { promptAction, router } from '@kit.ArkUI';
import { apiLogin, apiRegister, setToken, setUserId } from
'../common/services/ApiService';

@Entry
@Component
struct LoginPage {
  @State username: string = '';

```

```

@State password: string = '';
@State isLoading: boolean = false;
@State isRegisterMode: boolean = false;

async login(): Promise<void> {
  if (this.username.length === 0 || this.password.length === 0) {
    promptAction.showToast({ message: '请输入用户名和密码' });
    return;
  }
  this.isLoading = true;
  try {
    const result = await apiLogin(this.username, this.password);
    if (result.success) {
      setToken(result.data.token);
      setUserId(result.data.user.id);
      promptAction.showToast({ message: '登录成功' });
      router.pushUrl({
        url: 'pages/HomePage',
        params: { username: result.data.user.username, role:
result.data.user.role }
      });
    } else {
      promptAction.showToast({ message: result.message || '用户名
或密码错误' });
    }
  } catch (error) {
    promptAction.showToast({ message: '网络错误, 请重试' });
  } finally {
    this.isLoading = false;
  }
}

build() {
  Column() {
    Text('二手交易平台').fontSize(32).fontWeight(700).margin({ top:
100, bottom: 60 })
    TextInput({ placeholder: '请输入用户名', text: this.username })
      .width('80%').height(50).margin({ bottom: 20 })
      .onChange((value: string) => { this.username = value; })
    TextInput({ placeholder: '请输入密码', text: this.password })
      .type(InputType.Password).width('80%').height(50).margin({
bottom: 40 })
      .onChange((value: string) => { this.password = value; })
    Button(this.isLoading ? '登录中...' : '登录')
      .width('80%').height(50).fontSize(18).backgroundColor('#007AFF')
      .onClick(() => { this.login(); }).enabled(!this.isLoading)
      .width('100%').height('100%').backgroundColor('#F5F5F5')
  }
}

```

2.2 首页功能模块

```

// HomePage.ets - 首页核心
@Entry
@Component
struct HomePage {
  @State username: string = '';
  @State role: string = 'user';
  @State unreadNotificationCount: number = 0;

  private userModules: Array<Module> = [
    { id: 'productList', title: '逛逛商品', icon: '🛒', description: '发现心仪好物', route: 'pages/ProductList' },
    { id: 'aiAssistant', title: 'AI助手', icon: '🧠', description: '智能推荐与分析', route: 'pages/AiAssistant' },
    { id: 'cart', title: '购物车', icon: '🛒', description: '管理购物车商品', route: 'pages/Cart' },
    { id: 'friends', title: '好友管理', icon: '👤', description: '添加管理好友', route: 'pages/FriendList' },
    { id: 'credit', title: '信用评估', icon: '📊', description: 'AI智能信用分析', route: 'pages/CreditEvaluation' },
    { id: 'device', title: '设备协同', icon: '🔗', description: '多设备协同交易', route: 'pages/DeviceSynergy' },
    { id: 'publish', title: '发布商品', icon: '📦', description: '出售闲置物品', route: 'pages/Productization' },
    { id: 'security', title: '数据安全', icon: '🔒', description: '交易数据保障', route: 'pages/DataSecurity' },
    { id: 'profile', title: '个人中心', icon: '👤', description: '个人信息管理', route: 'pages/Profile' }
  ];

  private adminModules: Array<Module> = [
    // ...管理员特有功能：数据统计、用户管理、交易管理等
  ];
}

```

2.3 AI信用评估模块

```

// CreditEvaluation.ets - AI信用评估核心
import { apiCreditEvaluation, CreditEvaluationResult } from
'../common/services/ApiService';

@Entry
@Component
struct CreditEvaluationPage {

```

```

    @State creditScore: number = 0;
    @State evaluationResult: string = '';
    @State isLoading: boolean = true;
    @State riskLevel: string = '';
    @State selectedModel: string = 'glm';

    async evaluateCredit(): Promise<void> {
        this.isLoading = true;
        try {
            const result: CreditEvaluationResult = await
apiCreditEvaluation(this.selectedModel);
            if (result.success) {
                this.creditScore = result.data.score;
                this.evaluationResult = result.data.level;
                this.riskLevel = result.data.riskLevel;
                promptAction.showToast({ message: '信用评估完成' });
            }
        } catch (error) {
            promptAction.showToast({ message: '网络错误, 请重试' });
        } finally {
            this.isLoading = false;
        }
    }

    async switchModel(model: string): Promise<void> {
        this.selectedModel = model;
        await this.evaluateCredit();
    }
}

```

2.4 设备协同模块

```

// DeviceSynergy.ets - 设备协同核心
import { apiGetDeviceList, apiSyncTrade } from
'../common/services/ApiService';

@Entry
@Component
struct DeviceSynergy {
    @State devices: Array<Device> = [];
    @State loading: boolean = true;
    @State syncHistory: Array<SyncHistory> = [];

    async scanDevices(): Promise<void> {
        this.loading = true;
        try {
            const result = await apiGetDeviceList();
            if (result.success) {
                this.devices = result.data.map((d: DeviceInfo) => ({
                    id: d.device_id, name: d.device_name,
                    type: d.device_type, status: d.status,
                    synced: d.is_same_account && d.status === 'online',
                    lastSyncTime: '刚刚'
                }));
            }
        }
    }
}

```



```

        }));
    }
} catch (error) {
    promptAction.showToast({ message: '设备扫描失败' });
} finally {
    this.loading = false;
}
}

async toggleSync(device: Device): Promise<void> {
    device.synced = !device.synced;
    const result = await apiSyncTrade(device.id, device.synced ?
'1' : '0');
    if (result.success) {
        promptAction.showToast({ message: device.synced ? '已开启同步'
: '已关闭同步' });
    }
}
}
}

```

3. AI服务集成

3.1 多引擎AI策略

```

// AIService.ets - AI服务接口
export enum AIModelType {
    GLM = 'GLM',           // 智谱清言
    DEEPSEEK = 'DeepSeek', // 深度求索
    DOUBAO = 'DouBao'      // 豆包
}

async evaluateCredit(userId: string, historyData: object):
Promise<CreditEvaluationResult> {
    const prompt = `请分析以下用户的交易信用记录...`;

    // 多级降级策略：依次尝试三大模型
    try {
        const glmResult = await this.callGLM(prompt);
        if (glmResult.success) return glmResult;
    } catch (e) { /* 降级到DeepSeek */ }

    try {
        const deepseekResult = await this.callDeepSeek(prompt);
        if (deepseekResult.success) return deepseekResult;
    } catch (e) { /* 降级到豆包 */ }

    try {
        const doubaoResult = await this.callDouBao(prompt);
        if (doubaoResult.success) return doubaoResult;
    }
}

```

```

    } catch (e) { /* 降级到本地算法 */ }

    return this.generateMockResult();
  }
}

```

4. 后端API服务

4.1 Express服务架构

```

// server.js - 后端入口
const express = require('express');
const cors = require('cors');
const path = require('path');
const app = express();

app.use(cors());
app.use(express.json({ limit: '10mb' }));

// 路由模块 (12个)
app.use('/api', require('./routes/user'));           // 用户
app.use('/api', require('./routes/product'));        // 商品
app.use('/api', require('./routes/trade'));           // 交易
app.use('/api', require('./routes/cart'));            // 购物车
app.use('/api', require('./routes/device'));          // 设备协同
app.use('/api', require('./routes/ai'));              // AI服务
app.use('/api', require('./routes/message'));         // 消息
app.use('/api', require('./routes/friend'));          // 好友
app.use('/api', require('./routes/notification'));    // 通知
app.use('/api', require('./routes/address'));         // 地址
app.use('/api', require('./routes/favorite'));        // 收藏
app.use('/api', require('./routes/report'));          // 数据统计

app.listen(3000, () => {
  console.log('Server running on port 3000');
});

```

4.2 数据库设计 (SQLite)

```

-- 核心数据表 (共11张表)
CREATE TABLE users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT UNIQUE NOT NULL,

```

```

password TEXT NOT NULL,
email TEXT DEFAULT '',
phone TEXT DEFAULT '',
avatar TEXT DEFAULT '',
credit_score INTEGER DEFAULT 80,
role TEXT DEFAULT 'user',
created_at TEXT NOT NULL,
updated_at TEXT NOT NULL
);

CREATE TABLE products (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  price REAL NOT NULL,
  description TEXT DEFAULT '',
  image TEXT DEFAULT '',
  images TEXT DEFAULT '[]',
  category TEXT DEFAULT '其他',
  condition TEXT DEFAULT '二手',
  seller_id INTEGER NOT NULL,
  status TEXT DEFAULT 'active',
  view_count INTEGER DEFAULT 0,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL,
  FOREIGN KEY (seller_id) REFERENCES users(id)
);

CREATE TABLE trades (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  product_id INTEGER NOT NULL,
  buyer_id INTEGER NOT NULL,
  seller_id INTEGER NOT NULL,
  status TEXT DEFAULT 'pending',
  amount REAL NOT NULL,
  message TEXT DEFAULT '',
  address TEXT DEFAULT '',
  receiver_name TEXT DEFAULT '',
  receiver_phone TEXT DEFAULT '',
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL,
  FOREIGN KEY (product_id) REFERENCES products(id),
  FOREIGN KEY (buyer_id) REFERENCES users(id),
  FOREIGN KEY (seller_id) REFERENCES users(id)
);

CREATE TABLE credit_records (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL,
  score INTEGER NOT NULL,
  level TEXT NOT NULL,
  risk_level TEXT NOT NULL,
  analysis TEXT DEFAULT '',
  model TEXT DEFAULT 'GLM',
  evaluated_at TEXT NOT NULL,
  FOREIGN KEY (user_id) REFERENCES users(id)
);

CREATE TABLE cart (
  id INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

    user_id INTEGER NOT NULL,
    product_id INTEGER NOT NULL,
    quantity INTEGER DEFAULT 1,
    created_at TEXT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(id),
    FOREIGN KEY (product_id) REFERENCES products(id),
    UNIQUE(user_id, product_id)
);

CREATE TABLE messages (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    from_user_id INTEGER NOT NULL,
    to_user_id INTEGER NOT NULL,
    content TEXT NOT NULL,
    type TEXT DEFAULT 'text',
    product_id INTEGER DEFAULT NULL,
    is_read INTEGER DEFAULT 0,
    created_at TEXT NOT NULL,
    FOREIGN KEY (from_user_id) REFERENCES users(id),
    FOREIGN KEY (to_user_id) REFERENCES users(id)
);

CREATE TABLE addresses (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    name TEXT NOT NULL,
    phone TEXT NOT NULL,
    province TEXT NOT NULL,
    city TEXT NOT NULL,
    district TEXT,
    detail TEXT NOT NULL,
    is_default INTEGER DEFAULT 0,
    created_at TEXT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(id)
);

CREATE TABLE bargain_requests (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    product_id INTEGER NOT NULL,
    buyer_id INTEGER NOT NULL,
    seller_id INTEGER NOT NULL,
    original_price REAL NOT NULL,
    offered_price REAL NOT NULL,
    suggested_price REAL DEFAULT 0,
    message TEXT DEFAULT '',
    status TEXT DEFAULT 'pending',
    created_at TEXT NOT NULL,
    updated_at TEXT NOT NULL,
    FOREIGN KEY (product_id) REFERENCES products(id),
    FOREIGN KEY (buyer_id) REFERENCES users(id),
    FOREIGN KEY (seller_id) REFERENCES users(id)
);

CREATE TABLE notifications (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    title TEXT NOT NULL,
    content TEXT NOT NULL,
    type TEXT DEFAULT 'system',
    is_read INTEGER DEFAULT 0,

```

```
        created_at TEXT NOT NULL,
        FOREIGN KEY (user_id) REFERENCES users(id)
    );

CREATE TABLE favorites (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    product_id INTEGER NOT NULL,
    created_at TEXT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(id),
    FOREIGN KEY (product_id) REFERENCES products(id),
    UNIQUE(user_id, product_id)
);

CREATE TABLE friends (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    friend_id INTEGER NOT NULL,
    status TEXT DEFAULT 'pending',
    created_at TEXT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(id),
    FOREIGN KEY (friend_id) REFERENCES users(id),
    UNIQUE(user_id, friend_id)
);
```

后端采用 `sql.js` (SQLite的WebAssembly实现) 作为数据库引擎，无需安装额外的数据库服务，轻量便捷。

第四部分：测试验证与打包部署

1. 测试用例

测试项	预期结果	实际结果
应用启动	正常启动，显示登录页	[√] 成功
用户注册	输入信息注册新用户	[√] 成功
用户登录	正确用户名密码登录成功，错误则提示	[√] 成功
角色区分	管理员显示管理功能，用户显示交易功能	[√] 成功
商品列表展示	显示商品数据，支持分类筛选	[√] 成功
商品详情展示	显示商品完整信息	[√] 成功
AI信用评估	生成信用评分和风险等级	[√] 成功
设备协同同步	多设备状态同步	[√] 成功
购物车操作	添加、删除、修改数量	[√] 成功

测试项	预期结果	实际结果
好友聊天	发送接收消息	[√] 成功
商品发布	图片上传，信息保存	[√] 成功
通知推送	系统通知、交易通知	[√] 成功
数据统计	管理员查看平台数据总览	[√] 成功
AI议价	发起议价请求，AI给出建议	[√] 成功

2. 打包部署

平台	打包命令	部署方式
HarmonyOS	DevEco Studio → Build → Build HAP(s)	HAP安装包安装至鸿蒙设备
Node.js后端	<code>node src/server.js</code>	本地服务器运行，默认端口3000

第五部分：技术选型对比报告

1. 开发效率对比

技术栈	开发用时	代码量 (约)	难度评分
HarmonyOS (ArkTS)	80小时	6,500行	中等
Android (Java)	60小时	3,500行	中等
Flutter (Dart)	70小时	5,200行	中等
MAUI (C#)	75小时	7,500行	较高
Cordova (HTML/JS)	50小时	2,916行	较低
微信小程序	40小时	1,200行	较低

2. 性能对比

技术栈	启动速度	内存占用	UI流畅度
HarmonyOS	快 (<1s)	低 (<100MB)	流畅

技术栈	启动速度	内存占用	UI流畅度
Android	快 (<1s)	中 (~120MB)	流畅
Flutter	较快 (~1.5s)	中 (~110MB)	流畅
MAUI	较慢 (~2s)	较高 (~150MB)	一般
Cordova	较慢 (~2s)	高 (~180MB)	一般
微信小程序	快 (<1s)	低 (~80MB)	流畅

3. 适用场景分析

技术栈	适用场景	优势	劣势
HarmonyOS	鸿蒙生态设备、分布式场景	原生分布式能力、性能优异	生态相对较新
Android	全球Android设备	用户基数大、生态成熟	碎片化严重
Flutter	跨平台UI密集型应用	性能接近原生、热重载	包体积大
MAUI	.NET生态企业应用	C#语言、Visual Studio集成	社区较小
Cordova	快速原型、简单应用	开发效率高、Web技术栈	性能受限
微信小程序	微信生态内应用	获客成本低、即用即走	功能受限

4. 技术选型理由

选择HarmonyOS的原因：

- 鸿蒙是国产操作系统，作为计算机专业学生有必要掌握
- ArkTS声明式开发范式与React/Vue相似，学习成本低
- 原生分布式能力是其他平台不具备的差异化优势
- 二手交易平台需要设备协同、跨设备流转等场景，鸿蒙原生支持
- 华为生态持续发展，鸿蒙市场份额持续增长

第六部分：项目总结报告

1. 项目概述

- 项目名称：二手物品交易平台（CG2-SGTPDI）
- 开发目标：基于HarmonyOS实现分布式交易体验、AI信用评估、设备协同功能
- 团队规模：6人（分别负责6个技术栈）

- **个人负责**: HarmonyOS端全部开发 (22个页面+ 5个Service模块)
- **开发周期**: 15天
- **后端技术栈**: Node.js + Express + SQLite (sql.js)

2. 开发过程

- **需求分析** (第1-2天) : 确定二手交易平台核心功能需求, 设计数据库表结构
- **环境搭建** (第3天) : DevEco Studio安装, Node.js环境配置
- **核心开发** (第4-10天) : 完成22个页面开发, 实现登录、商品展示、AI评估、设备协同等全部功能
- **后端开发** (第8-10天) : Express API开发, SQLite数据库接口实现
- **系统整合** (第11-12天) : 前后端联调, 功能测试
- **测试交付** (第13-15天) : 功能测试、兼容性测试、文档编写

3. 个人贡献统计

阶段	完成功能	代码行数	工时	贡献度
第一阶段	项目初始化、登录注册、首页框架	3,200行	15h	30%
第二阶段	商品浏览、购物车、交易下单、AI信用评估、设备协同、好友聊天、通知、商品发布	5,600行	40h	25%
第三阶段	AI议价、数据安全、数据统计、用户管理、地址管理、数据库查看、个人中心、测试优化	3,000行	25h	20%

4. 特色功能实现

分布式交易体验

基于HarmonyOS分布式能力, 通过DistributedService.ets实现跨设备数据同步。使用@ohos.distributedKVStore分布式数据库, 同账号设备间自动同步浏览记录、购物车和交易状态。用户可在手机、平板、智慧屏间无缝切换操作。

AI信用评估

调用GLM (智谱清言)、DeepSeek (深度求索)、豆包 (字节跳动) 三大国产AI模型对买卖双方的历史交易记录、评价数据进行了综合分析, 自动生成信用评分和交易风险提示。采用多级缓存策略, 确保服务高可用性。

AI智能议价

在商品详情页集成AI议价功能，用户输入期望价格后，AI自动分析商品市场行情并给出建议价格，辅助买卖双方达成交易。

设备协同

实现多设备间的消息通知同步和交易状态实时更新，支持手机、平板、智慧屏等多设备协同。用户可一键开启/关闭设备同步，实时查看同步历史记录。采用deviceManager API 获取同账号设备列表，通过filter 筛选is_same_account 设备。

5. 收获与体会

- **技术成长**：掌握了HarmonyOS应用开发，熟练使用ArkTS声明式UI和装饰器（@State/@Prop/@Link）状态管理
- **Git协作**：体验了Git团队协作流程，掌握分支管理和代码提交流程
- **AI工具使用**：学会了使用TRAE AI辅助开发，提高了开发效率约50%
- **分布式能力**：深入理解了HarmonyOS分布式能力的应用场景和实现方式
- **问题解决**：通过多级降级策略解决了AI模型调用失败时的服务可用性问题

6. 问题与改进

- **遇到的问题**：
 - AI模型API调用失败时的降级处理（通过多级降级策略解决）
 - 设备协同API返回空列表（通过onForeground回调自动同步解决）
 - ohpm依赖版本冲突（移除冗余依赖，使用本地内置插件）
- **改进方向**：
 - 优化AI模型调用性能，减少响应时间
 - 增加更多分布式功能 如跨设备文件传输
 - 完善离线模式支持
 - 引入Redis缓存提升热点数据访问速度

实验结果

验证结果

技术栈	应用运行	功能完整	性能表现
HarmonyOS	[x] 成功	[x] 成功	[x] 良好

截图记录

登录页面

首页

AI信用评估

考核指标

考核项	评分标准	得分
功能完整性	各技术栈应用功能完整	20分
代码质量	代码结构清晰，注释完整	15分
团队协作	Git流程规范，协作良好	20分
多端实现	完成HarmonyOS开发	25分
技术对比	完成技术选型分析报告	10分
实验报告	报告结构完整，内容详实	10分

教师评价

项目	评价
实验完成情况	
技术掌握程度	
团队协作	
创新点	分布式交易体验、AI信用评估、设备协同
综合评价	
成绩	

教师签名：_____

日期：_____